

Agile Project Documentation

Title

**AI Career Companion Agent for Internship Matching and
Interview Preparation**

Project Title:

TalentSprint AI

Prepared By:

Bharath Kumar J M

Project Duration:

22/07/2027 – 22/09/2027

Mentor:

Vijay Parmar

Index

No	Section	Page No
1	Project Overview	03
2	Project Objectives	04
3	Technology Stack	05
4	Project Requirements	06
5	System Architecture	07
6	Product Backlog	10
7	Sprint Planning	12
8	Sprint Tasks & Status	15
9	Implementation	17
10	Testing	24
11	Challenges & Solutions	28
12	Final Project Status	32

1. Project Overview

1.1 Project Name

TalentSprint AI

1.2 What is the Project?

The TalentSprint AI is an AI-powered platform designed to help students find and apply for suitable internships. It allows students to create their profile, manage resumes, analyze their skills, match with relevant internship opportunities, generate cover letters, prepare for interviews, track applications, and interact with an AI assistant for personalized career guidance.

1.3 Problem Statement

Students often face difficulty finding **relevant internship opportunities** that match their education, skills, interests, and career goals. Internship platforms may provide many opportunities, but students still have to manually search through them and determine whether they are suitable.

There are also several related challenges:

- Students may not know **which internships match their skills**.
- They may not clearly understand their **skill gaps**.
- Preparing a suitable **resume and cover letter** for different internships takes time.
- Students may struggle with **interview preparation**.
- Managing and tracking multiple internship applications can become difficult.
- Students may need personalized guidance about **career paths and suitable job roles**.

In one sentence

The problem is the lack of a personalized system that can intelligently connect students' profiles, skills, resumes, and career goals with suitable internship opportunities while also supporting them throughout the internship application process.

1.4 Target Users

- **Students & Internship Seekers:** Find suitable internships and receive AI-based career guidance.
- **Fresh Graduates:** Discover career opportunities and identify required skills.
- **College Placement Teams:** Support students in internship and career preparation.

2. Project Objective

The primary objective of **TalentSprint AI** is to develop an AI-powered platform that simplifies the internship search and career preparation process for students and internship seekers. The system aims to understand a user's profile, skills, resume, and career interests and provide personalized assistance throughout the internship journey.

Objectives

1. Find Suitable Internship Opportunities

TalentSprint AI aims to help students discover internship opportunities that are relevant to their educational background, technical skills, interests, and career goals.

2. Analyze Resumes and Identify Skill Gaps

The system analyzes the student's resume and existing skills to understand their current capabilities. It identifies missing or weak skills that may affect their suitability for specific internship roles and helps students understand areas where improvement is required.

3. Provide Personalized Career Guidance

TalentSprint AI provides personalized career guidance based on the student's profile, skills, interests, and goals. It can recommend suitable career paths, job roles, and development areas to help students make better career decisions.

4. Provide Learning and Skill Improvement Recommendations

The platform identifies the skills required for desired internship or career roles and recommends relevant areas for learning and improvement. This helps students build the necessary technical and professional skills before applying for opportunities.

5. Generate Personalized Cover Letters

The system helps students create suitable cover letters based on their profile, resume, skills, and the selected internship opportunity. This reduces the effort involved in preparing individual applications.

6. Support Interview Preparation

TalentSprint AI aims to help students prepare for internship interviews by providing role-related questions, preparation guidance, and personalized recommendations based on the selected internship or career path.

7. Track Internship Applications

The platform provides application tracking capabilities so that students can organize and monitor their internship applications, helping them keep track of application progress and opportunities.

8. Provide an AI-Based Career Assistant

The system provides an AI assistant that allows students to interact with the platform, ask questions, receive career-related guidance, and get personalized assistance throughout their internship and career preparation journey.

3. Technology Stack

Category	Technology
Frontend	React 19
Build Tool	Vite 8
Routing	React Router DOM 7
HTTP Client	Axios
Styling	Vanilla CSS3
Backend	Python 3.10+ / 3.14
Backend Framework	FastAPI
Server	Uvicorn
Data Validation	Pydantic v2
Authentication	JWT / python-jose
Password Security	Passlib / bcrypt
Database	PostgreSQL
ORM	SQLAlchemy
Vector Search	pgvector
AI / LLM	Groq API
LLM Models	GPT-OSS-120B, GPT-OSS-20B, Groq Compound
NLP	Custom NLP Pipeline
RAG	Retrieval-Augmented Generation
PDF Processing	PyMuPDF, pypdf
DOCX Processing	python-docx
PDF Generation	ReportLab
Version Control	Git / GitHub

4. Project Requirements

4.1 Functional Requirements

1. User Authentication & Profiles -The system shall provide secure registration, login, password recovery, and JWT-based authentication. Users can create and manage profiles containing education, skills, projects, experience, and career interests.

2. Document Upload & Ingestion - Users shall be able to upload PDF, DOCX, and TXT resumes with file type and size validation. Uploaded documents are processed for further analysis.

3. Document Parsing & ATS Audit - The system shall extract skills, education, projects, and experience from resumes. It shall also perform ATS analysis including keyword, action-verb, and readability checks.

4. Intelligent Querying & Voice Interaction - Users can interact with the AI assistant through text or voice. The system supports conversational queries and quick prompts related to internships, careers, resumes, and skills.

5. Context-Grounded AI Responses - The AI shall generate responses using relevant information from the user's resume and uploaded documents. RAG is used to retrieve relevant context, with support for direct and detailed responses.

6. Chat History & Export - The system shall store chat history and allow users to copy or export AI responses in PDF, Word, Markdown, and PNG formats.

4.2 Non-Functional Requirements

1. Performance - The system shall provide fast responses using asynchronous FastAPI APIs, optimized frontend builds, and Groq-based AI inference.

2. Security & Privacy - User data shall be protected using JWT authentication, bcrypt password hashing, input validation, CORS controls, and secure database queries.

3. Scalability - The application shall support increasing users and requests through asynchronous FastAPI, PostgreSQL, and pgvector.

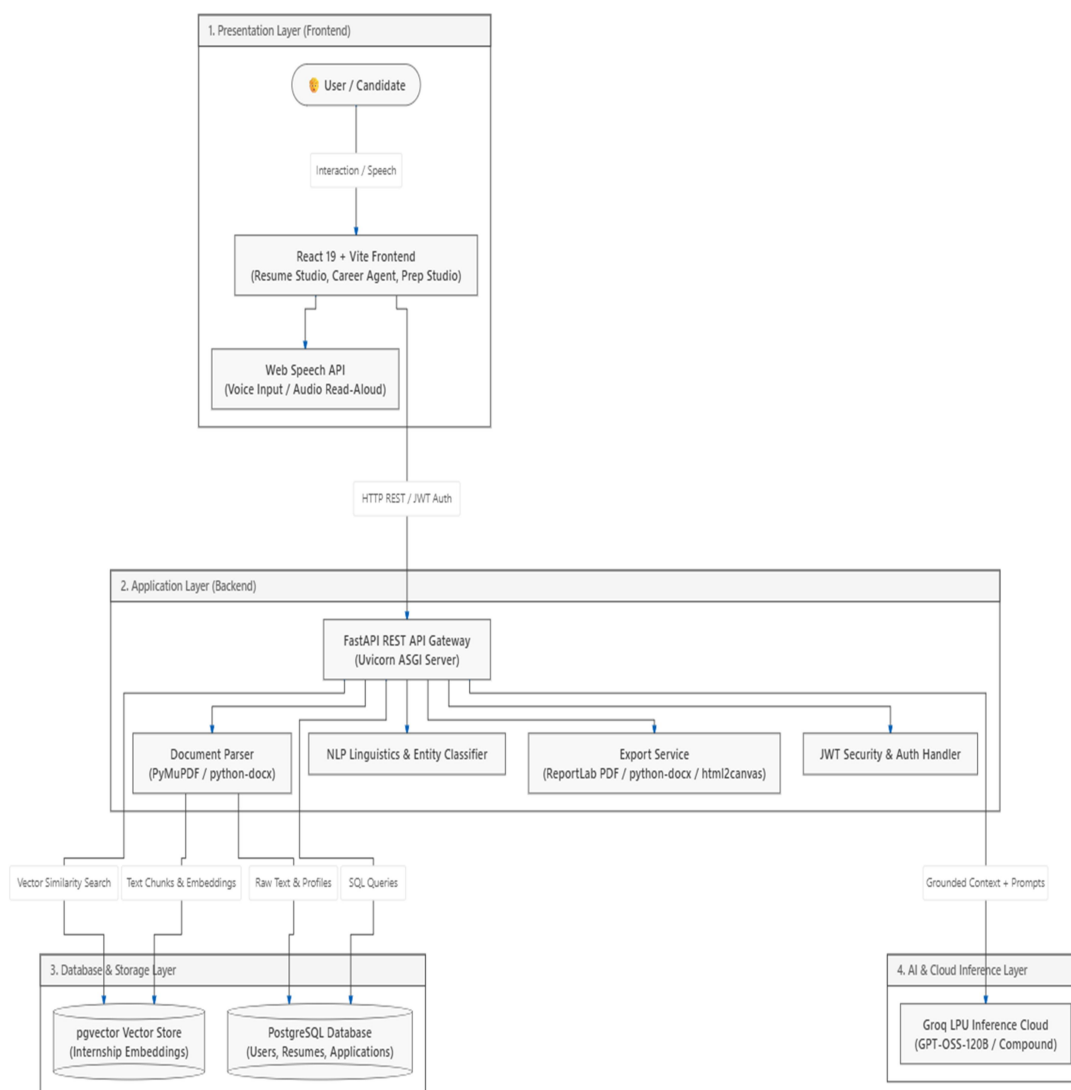
4. Usability - The system shall provide a responsive and user-friendly interface with simple navigation, upload features, AI chat, and feedback notifications.

5. Reliability - The system shall handle API, upload, and processing failures gracefully and provide fallback document processing when AI services are unavailable.

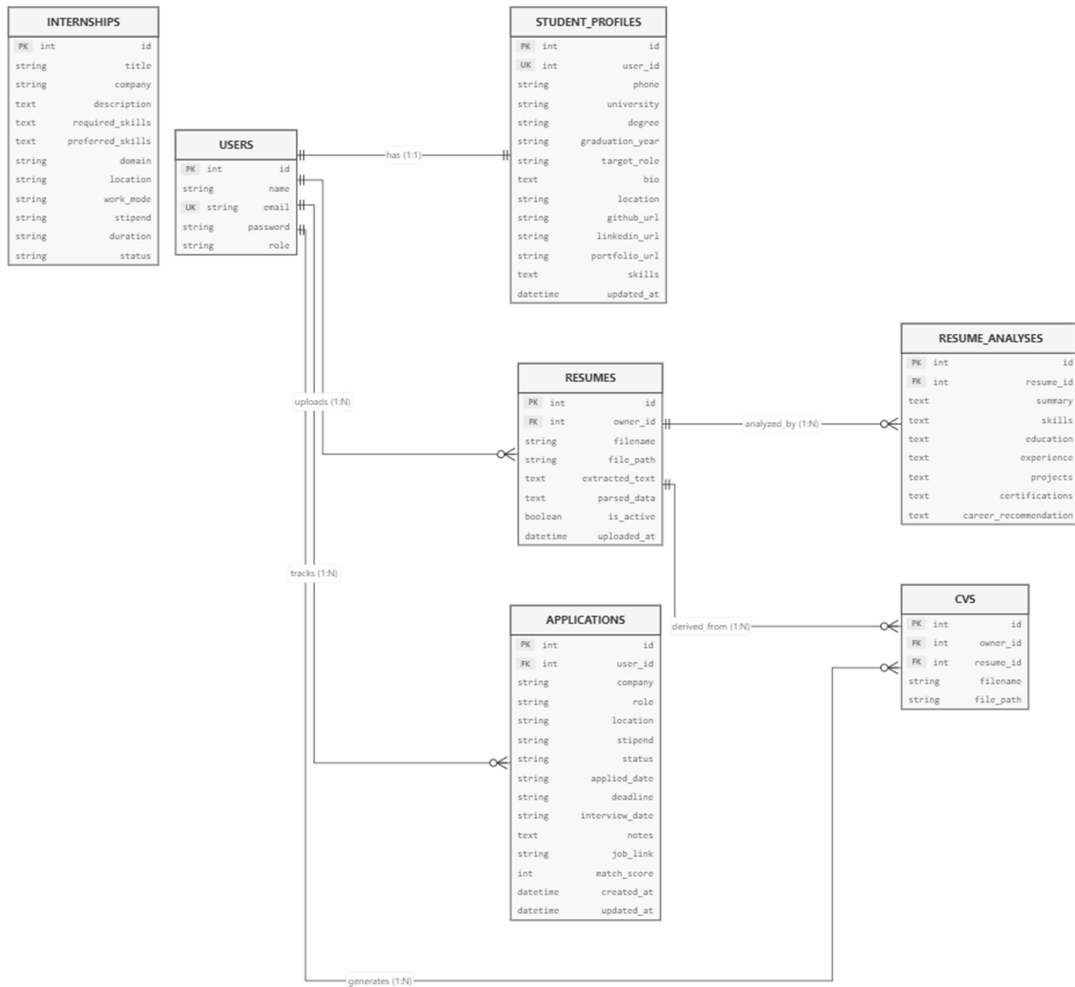
5. System Architecture

The **TalentSprint AI system architecture** is designed as a layered architecture where each layer performs a specific role. The **Presentation Layer** uses React to provide the user interface, dashboards, resume upload, and AI chat features. The **Application Layer** uses FastAPI to handle authentication, API requests, document processing, and application logic. The **Data Layer** uses PostgreSQL to store user profiles, resumes, applications, and related information. The **AI & RAG Layer** uses pgvector for semantic search and Groq LLMs to generate personalized, context-aware responses.

5.1 System Architecture Flow Diagram



5.2 Database Schema & Entity-Relationship (ER) Diagram



5.3 Main System Components

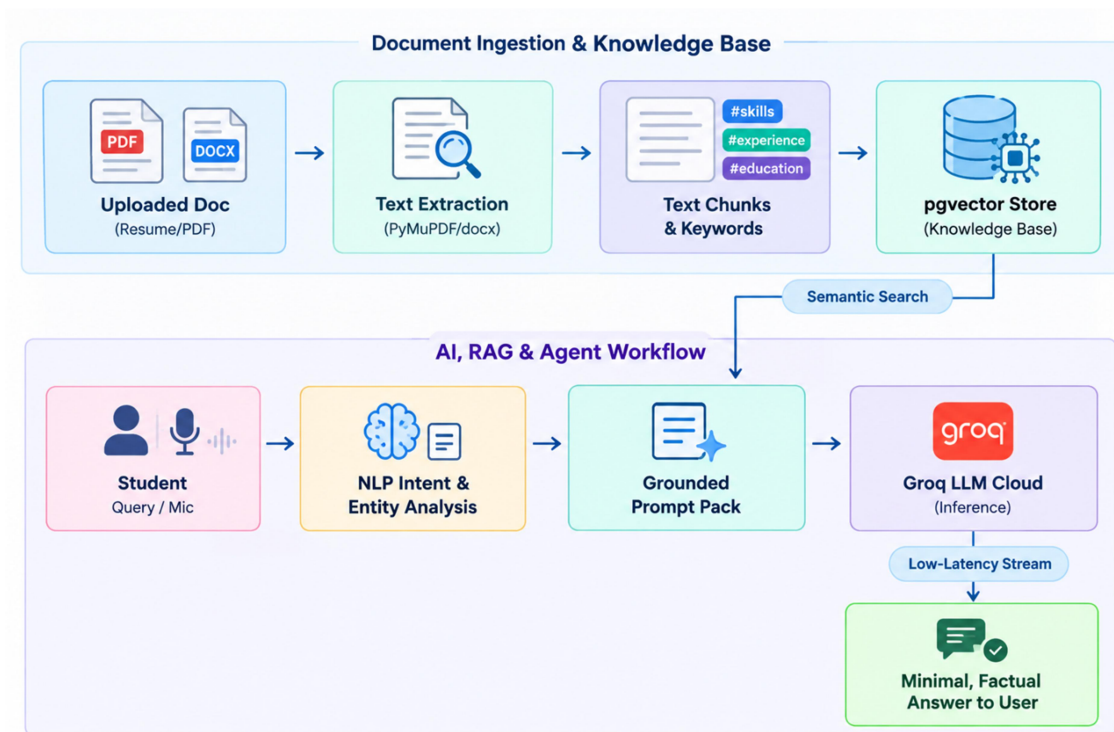
1. **Presentation Layer (React 19 + Vite):** Provides the user interface, dashboards, AI chat, voice interaction, and export features.
2. **Application Layer (FastAPI + Python):** Handles API requests, authentication, document processing, NLP analysis, and application logic.
3. **Data Layer (PostgreSQL & pgvector):** Stores user profiles, resumes, application data, and vector embeddings for similarity-based search.
4. **AI & Inference Layer (Groq Cloud):** Processes user queries and generates fast, personalized AI responses using LLMs.

5.4 End-to-End API & Data Flow

1. **User Action:** The user submits an AI query or uploads a resume through the React frontend.
2. **FastAPI Gateway:** The backend verifies authentication and routes the request to the appropriate service.

3. **Document Processing:** Uploaded documents are extracted, analyzed for ATS compatibility, and stored in the database.
4. **Candidate Context:** For AI queries, the system retrieves the user's resume and relevant profile information.
5. **Groq LLM Processing:** The query and relevant candidate context are sent to the Groq LLM for response generation.
6. **Response Generation:** The AI produces a concise and context-aware response based on the retrieved information.
7. **Frontend Response:** The React application displays the response and updates the user's chat history.

5.5 AI, RAG & Agent Workflow



1. **Document Ingestion:** Uploaded resumes are converted into text and organized into sections such as skills, education, experience, and projects.
2. **Context Grounding:** The system retrieves relevant candidate information and internship data using semantic search.
3. **NLP Analysis:** The user's query is analyzed to identify intent, entities, and relevant career information.
4. **Prompt Generation:** The retrieved information is combined with the user's query to create a context-aware prompt for the LLM.
5. **AI Inference:** Groq LLM processes the grounded prompt and generates a personalized response.

6. Product Backlog

The Product Backlog contains and prioritizes the core features, technical milestones, and development tasks required to build the **TalentSprint AI** career platform. Each backlog item is assigned a priority and development status to support effective sprint planning and progress tracking.

6.1 Backlog Item Summary

ID	Feature / Task	Priority	Status
P001	Project Setup & Environment Architecture – Initialize the React 19 + Vite frontend, FastAPI backend, and project environment configuration.	High	Done
P002	Database Setup & Schema Modeling – Configure PostgreSQL with SQLAlchemy models and the pgvector extension for vector storage.	High	Done
P003	User Authentication & Authorization – Implement JWT-based authentication, bcrypt password hashing, and password recovery functionality.	High	Done
P004	Document Upload & Parsing Engine – Support PDF, DOCX, and TXT document uploads with text extraction and validation.	High	Done
P005	AI Resume Parser & ATS Audit – Extract skills, experience, projects, keywords, and action verbs and evaluate ATS readiness.	High	Done
P006	RAG Pipeline & Semantic Knowledge Base – Implement pgvector-based semantic search to support internship recommendations and context-aware AI responses.	High	Done
P007	Interactive Chat Agent Interface – Develop the AI chat interface with Markdown formatting, code rendering, and quick-action options.	High	Done
P008	Voice Interaction (STT & TTS) – Integrate the Web Speech API for voice input through the microphone and AI response read-aloud functionality.	Medium	Done

ID	Feature / Task	Priority	Status
P009	Minimal Response Engine & Brevity Controls – Implement Minimal and Detailed response modes to control AI response length and reduce unnecessary content.	High	Done
P010	Multi-Format Session Export – Enable chat transcript export in PDF, Word, Markdown, and PNG formats.	Medium	Done
P011	Job Tracker & Skill Gap Analysis – Provide internship application tracking and role-specific skill gap analysis.	Medium	Done
P012	System Integration, Performance & Security Testing – Perform API, integration, performance, authentication, and error-handling tests.	High	In Progress
P013	Production Deployment & Cloud CI/CD – Prepare Docker containerization, cloud database configuration, CI/CD, and production deployment.	Medium	To Do

6.2 Priority & Status Legend

Priority Levels

- **High:** Critical infrastructure, authentication, core AI, database, and essential platform features.
- **Medium:** Supporting features, user-experience improvements, productivity tools, and deployment activities.
- **Low:** Optional enhancements, aesthetic improvements, and future integrations.

Status Definitions

- **To Do:** Planned and defined in the product backlog but development has not started.
- **In Progress:** Currently under development or implementation.
- **Testing:** Development is complete and the feature is undergoing verification and testing.
- **Done:** Feature has been implemented and verified in the working system.

7. Sprint Planning

The **TalentSprint AI** project was developed using the **Agile Scrum Methodology**, dividing the development lifecycle into **four focused, time-boxed sprints of one week each**, resulting in a total development period of **four weeks**. Each sprint focused on a specific set of features and delivered an incremental, testable component of the overall system.

Sprint 1: Project Setup & Infrastructure Initialization

Duration: 1 Week – Week 1

Sprint Goal: Establish the project foundation, development environment, application architecture, and database structure.

Sprint Backlog Tasks

1. **Requirement Analysis:** Identify and finalize the functional and non-functional requirements of the TalentSprint AI platform.
2. **Project Scaffolding:** Create the project structure with separate **React + Vite frontend** and **FastAPI backend** components.
3. **Version Control Setup:** Configure Git, project branching practices, .gitignore, and environment configuration.
4. **Database Architecture:** Configure **PostgreSQL** and create the required database models for users, documents, chat sessions, messages, and application data.
5. **Vector Database Setup:** Configure the **pgvector** extension to support semantic search and RAG-based functionality.
6. **Environment Configuration:** Create the Python virtual environment and install the required backend and frontend dependencies.

Deliverables / Milestones

- Working React frontend and FastAPI backend.
- PostgreSQL database with required schemas and models.
- Initial pgvector configuration for semantic search.
- Configured local development environment.

Sprint 2: Authentication & Document Processing

Duration: 1 Week – Week 2

Sprint Goal: Develop secure user authentication, profile management, and document upload and processing capabilities.

Sprint Backlog Tasks

1. **User Authentication:** Implement user registration and login using **JWT-based authentication**.
2. **Password Security:** Implement secure password hashing using **bcrypt** and password recovery functionality.

3. **Route Protection:** Protect private API endpoints using authentication dependencies and current-user validation.
4. **Document Upload:** Develop document upload functionality supporting **PDF, DOCX, and TXT** files with file type and size validation.
5. **Document Processing:** Extract text and relevant information from uploaded documents using **PyMuPDF, pypdf, and python-docx**.
6. **Data Persistence:** Store user profiles, document metadata, and processed information in PostgreSQL.
7. **API Verification:** Test and verify backend endpoints using FastAPI's interactive **Swagger UI**.

Deliverables / Milestones

- Functional user registration and login system.
- Secure JWT-protected API endpoints.
- Working document upload and text extraction pipeline.
- Resume and document information stored in PostgreSQL.

Sprint 3: AI Engine, RAG & Intelligent Career Features

Duration: 1 Week – Week 3

Sprint Goal: Implement the AI-powered resume analysis, semantic search, RAG pipeline, and personalized career assistance features.

Sprint Backlog Tasks

1. **Resume Analysis:** Develop AI-based resume parsing to identify skills, education, projects, experience, keywords, and other relevant information.
2. **ATS Audit:** Implement ATS analysis including keyword relevance, action verbs, readability, and resume readiness evaluation.
3. **Semantic Search:** Store and retrieve vector representations using **pgvector** for similarity-based information retrieval.
4. **RAG Pipeline:** Connect user queries with relevant resume, profile, and internship context before generating responses.
5. **LLM Integration:** Integrate the **Groq API** with supported LLM models to generate personalized career and internship responses.
6. **NLP Processing:** Implement intent, entity, tone, and career-related query analysis using the custom NLP pipeline.
7. **AI Career Features:** Implement career guidance, skill-gap analysis, internship matching, interview preparation, and cover-letter generation.
8. **Response Controls:** Implement Minimal and Detailed response modes to control the level of AI-generated information.

Deliverables / Milestones

- Functional AI resume analysis and ATS audit.
- Working pgvector-based semantic search.
- End-to-end RAG pipeline.
- Personalized AI career assistance and internship recommendations.

- Context-aware AI responses based on candidate information.

Sprint 4: Frontend Integration, Testing & Final Demonstration

Duration: 1 Week – Week 4

Sprint Goal: Integrate all backend and AI services with the React frontend, complete interactive features, perform system testing, and prepare the final demonstration.

Sprint Backlog Tasks

1. **Frontend Implementation:** Develop responsive interfaces for Login, Registration, Dashboard, Resume Analysis, AI Chat, Career Guidance, and Application Tracking.
2. **API Integration:** Connect React components with FastAPI REST APIs using **Axios** and JWT bearer-token authentication.
3. **AI Chat Integration:** Integrate the AI chat interface with Markdown rendering, quick actions, conversation history, and context-aware responses.
4. **Voice Interaction:** Integrate the **Web Speech API** for Speech-to-Text input and Text-to-Speech response playback.
5. **Export Features:** Implement conversation and document export in PDF, DOCX, Markdown, and PNG formats.
6. **Application Tracking:** Integrate internship application tracking and skill-gap analysis features.
7. **System Testing:** Perform API, integration, authentication, document-processing, UI, and error-handling tests.
8. **Performance & Security Verification:** Verify response performance, JWT validation, input validation, file restrictions, and protected routes.
9. **Demo Preparation:** Prepare sample resumes, verify major workflows, optimize the application, and prepare the final project demonstration.

Deliverables / Milestones

- Fully integrated full-stack TalentSprint AI application.
- Responsive and functional user interface.
- Integrated AI, RAG, resume analysis, and career assistance features.
- Completed system testing and validation.
- Application ready for final demonstration and evaluation.

8. Sprint Tasks & Status

To monitor development progress and maintain transparency throughout the project, all development tasks were tracked using a standardized Agile workflow:

To Do → In Progress → Testing / Review → Done

- **To Do:** Tasks planned for the sprint but not yet started.
- **In Progress:** Tasks currently under development or implementation.
- **Testing / Review:** Completed development tasks undergoing testing, verification, or review.
- **Done:** Tasks fully implemented, tested, reviewed, and accepted according to the project's Definition of Done (DoD).

8.1 Detailed Sprint Task Tracking

Sprint	Task ID	Task Description	Priority	Status
Sprint 1	T101	Requirement Gathering & System Specification	High	Done
Sprint 1	T102	Project Scaffolding & Git Repository Setup	High	Done
Sprint 1	T103	Database Architecture & SQLAlchemy Model Development	High	Done
Sprint 1	T104	Development Environment & Dependency Configuration	Medium	Done
Sprint 2	T201	User Registration & Login REST APIs	High	Done
Sprint 2	T202	JWT Authentication & Protected Route Implementation	High	Done
Sprint 2	T203	Multi-Format Document Upload (PDF/DOCX/TXT)	High	Done
Sprint 2	T204	Base UI Wireframes & Layout Development	Medium	Done
Sprint 3	T301	Resume Text Extraction & Normalization	High	Done
Sprint 3	T302	Text Chunking & Semantic Context Preservation	High	Done
Sprint 3	T303	Vector Embedding Generation & pgvector Indexing	High	Done
Sprint 3	T304	Groq LLM Integration & Prompt Engineering	High	Done
Sprint 3	T305	End-to-End RAG Query & Context Grounding Pipeline	High	Done

Sprint	Task ID	Task Description	Priority	Status
Sprint 4	T401	React Frontend & FastAPI API Integration using Axios	High	Done
Sprint 4	T402	Speech-to-Text Voice Query Feature	Medium	Done
Sprint 4	T403	Automated ATS Scoring & Keyword Matching	Medium	Done
Sprint 4	T404	CV Builder & PDF Generation using ReportLab	High	Done
Sprint 4	T405	Comprehensive System Testing & Bug Fixing	High	Done
Sprint 4	T406	Final Project Demonstration & Documentation	High	Done

8.2 Sprint Status Summary

Sprint	Major Focus	Total Tasks	Completed	Pending
Sprint 1	Project Setup & Infrastructure	4	4	0
Sprint 2	Authentication & Document Processing	4	4	0
Sprint 3	AI, RAG & LLM Integration	5	5	0
Sprint 4	Frontend, Testing & Finalization	6	6	0
Total	Complete Project Development	19	19	0

Overall Status

All planned tasks across the four development sprints have been marked as **Done**, covering project setup, authentication, document processing, AI and RAG implementation, frontend integration, testing, and final documentation.

9. Implementation & Key Features

This section describes the major features implemented in the **TalentSprint AI** platform, including their purpose, implementation workflow, technical components, and expected outputs.

Feature 1: User Authentication & Session Security

1. Description

The User Authentication module provides secure candidate registration, login, password protection, and session management. After successful registration, the user can proceed to the login interface and access protected platform features after authentication.

2. Implementation & Workflow

1. **Password Hashing:** User passwords are securely hashed using **bcrypt** through Passlib before being stored in the database. Plain-text passwords are never stored.
2. **JWT Token Generation:** After successful login, the backend generates a signed **JSON Web Token (JWT)** using `python-jose` with an expiration period.
3. **Route Protection:** FastAPI authentication dependencies validate JWT tokens and identify the authenticated user before allowing access to protected endpoints.
4. **Frontend Authentication:** The React frontend stores the authentication token and attaches it to subsequent API requests through the `Authorization: Bearer <token>` header.

3. Workflow



4. Output

Endpoint: POST `/api/auth/login`

Sample Response:

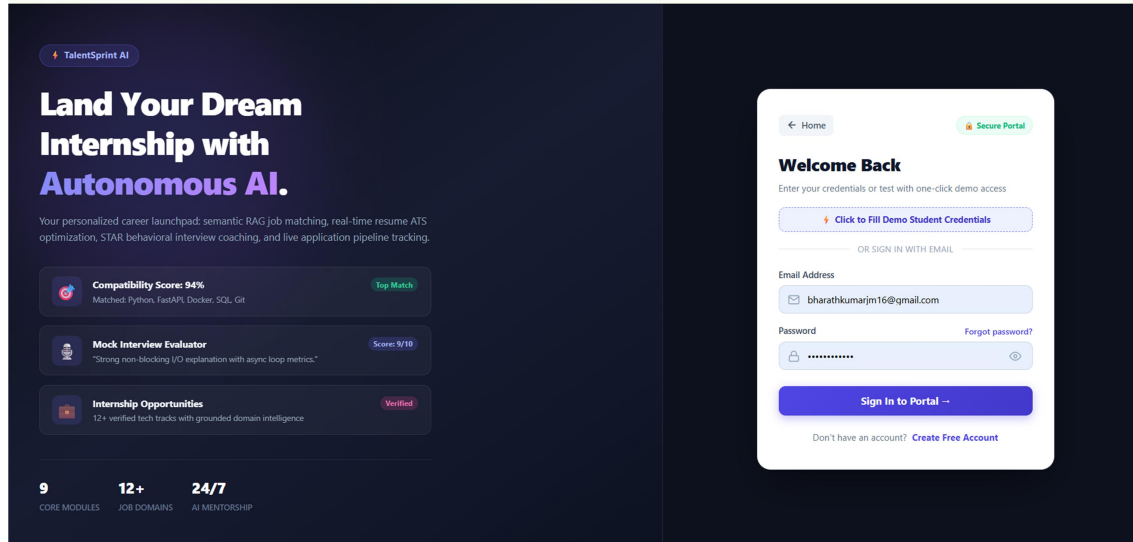
```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": 2,
    "email": "candidate@example.com",
```

```

    "full_name": "Candidate Name"
  }
}

```

Screenshot of Login Page:



Feature 2: Multi-Format Document Upload & Ingestion Pipeline

1. Description

The Document Upload module allows candidates to upload resumes and other career-related documents in **PDF, DOCX, and TXT** formats. Uploaded files are validated, processed, and stored for subsequent resume analysis and AI-based retrieval.

2. Implementation & Workflow

1. **File Upload:** FastAPI receives uploaded documents through an UploadFile endpoint.
2. **File Validation:** The system validates supported file formats, MIME types, and file-size limits before processing.
3. **Metadata Storage:** Document information such as filename, file size, upload time, and user ownership is stored in PostgreSQL.
4. **Text Extraction:** PDF, DOCX, and TXT files are processed using the appropriate document-processing libraries.
5. **Text Preparation:** Extracted text is cleaned and prepared for resume analysis and semantic retrieval.
6. **Chunking:** Large text content is divided into smaller sections to support efficient semantic search and RAG processing.

3. Workflow

```

Document Upload
  ↓
File Type & Size Validation

```

↓
Text Extraction
↓
Text Cleaning & Processing
↓
Text Chunking
↓
Database / Vector Storage

4. Output

Endpoint: POST /api/documents/upload

Sample Response:

```
{
  "message": "File uploaded and processed successfully",
  "document_id": 14,
  "filename": "Candidate_Resume.pdf",
  "file_size_kb": 128.4,
  "chunks_indexed": 8,
  "status": "Ready for RAG Q&A"
}
```

Screenshot of Profile & Resume:

The screenshot displays the 'Career Companion' web application. The main section is titled '1. Student Profile & Resume Management' and includes a sub-header 'Configure your academic background, target role, and manage resume versions'. A green notification bar at the top states 'Resume uploaded & parsed successfully: Bharath Kumar J M.pdf'. The interface is divided into three main columns. The left column contains a sidebar with navigation links: Dashboard, Profile & Resumes (active), Resume Parser, Internship Opportunities, Job Matching, Skill Gap Analysis, Resume & CV Builder, Interview Prep, Application Pipeline, and AI Career Companion. The middle column, titled 'Student Academic & Career Profile', contains form fields for personal and academic information: Full Name (Bharath kumar jm), Email Address (bharathkumarjm16@gmail.com), Phone Number (+1 (555) 382-9410), Current Location (San Francisco, CA), University / College (Computer Science & Engineering), Degree & Major (Bachelor of Engineering / B.Tech), Graduation Year (2026), Target Internship Role (Software Engineer Intern), and Professional Bio / Objective (Passionate student seeking opportunities to apply core engineering and development skills). It also includes fields for LinkedIn URL, GitHub URL, and Portfolio / Personal Website. The right column, titled 'Resume Management & Versions', shows an 'Upload New Resume' section with a 'Choose File' button and an 'Upload & Parse Resume' button. Below this, it lists 'Your Resume Versions' with two entries: 'Bharath Kumar J M.pdf' (uploaded Sep 15, 2026 16:55) and 'Bharath Kumar J M.docx' (uploaded Sep 11, 2026 09:16). Each entry includes a 'Deep Extract' button and a 'Download' button. The user's name 'Bharath kumar jm' and a 'Sign Out' button are visible in the top right corner.

Feature 3: AI-Powered Context-Grounded Chatbot & RAG Pipeline

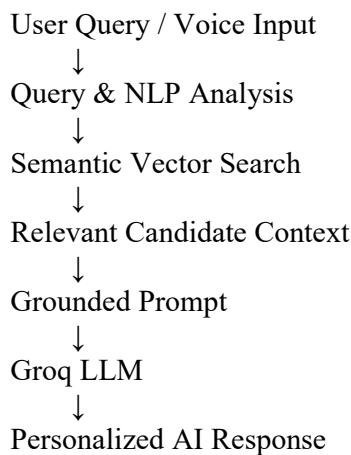
1. Description

The AI Chatbot allows candidates to ask questions related to their resume, skills, career opportunities, interview preparation, and skill gaps. The system uses **Retrieval-Augmented Generation (RAG)** to retrieve relevant candidate information and provide context-aware responses.

2. Implementation & Workflow

1. **Document Embedding:** Processed document chunks are converted into vector representations for semantic retrieval.
2. **Query Processing:** When a user submits a question, the query is analyzed and prepared for semantic retrieval.
3. **Vector Similarity Search:** Relevant information is retrieved from the **pgvector** knowledge base based on semantic similarity.
4. **Context Grounding:** Retrieved candidate information is combined with the user's question to create a context-aware prompt.
5. **LLM Processing:** The grounded prompt is sent to the **Groq LLM API** to generate a personalized response.
6. **Voice Interaction:** The Web Speech API allows users to provide voice input and receive spoken responses where supported.

3. Workflow



4. Sample Interaction

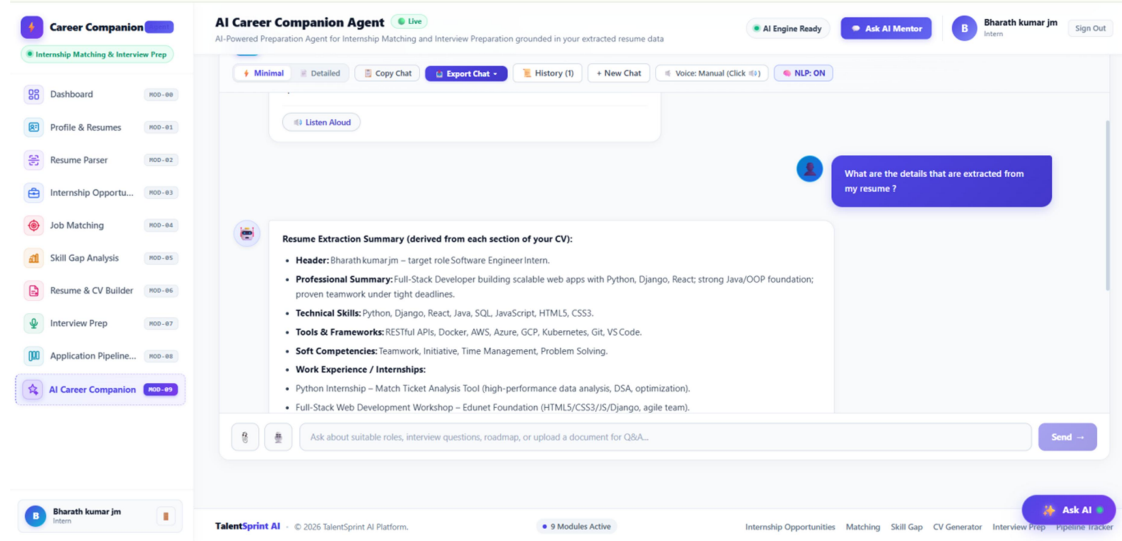
Sample Query:

"Which backend technologies are mentioned in my resume?"

AI Response:

The AI identifies relevant technologies from the candidate's uploaded resume and provides a context-based response.

Screenshot of AI assistant:



Feature 4: Automated ATS Resume Scorer & Analyzer

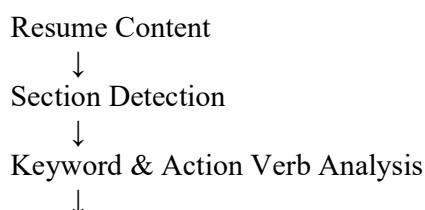
1. Description

The ATS Resume Analyzer evaluates the candidate's resume using automated checks for important resume sections, keywords, action verbs, readability, and overall ATS readiness. The results are presented as a score with improvement recommendations.

2. Implementation & Workflow

1. **Resume Section Detection:** The system identifies important sections such as Summary, Experience, Skills, Education, and Projects.
2. **Keyword Analysis:** Technical and job-related keywords are identified and evaluated for relevance.
3. **Action Verb Analysis:** The system checks for effective action verbs used to describe candidate achievements and responsibilities.
4. **Readability Analysis:** Resume content is analyzed for clarity and structural readability.
5. **Score Calculation:** Individual evaluation results are combined to generate an overall ATS readiness score.
6. **Recommendations:** The system provides suggestions for improving weak areas of the resume.

3. Workflow



Readability Evaluation



Score Calculation



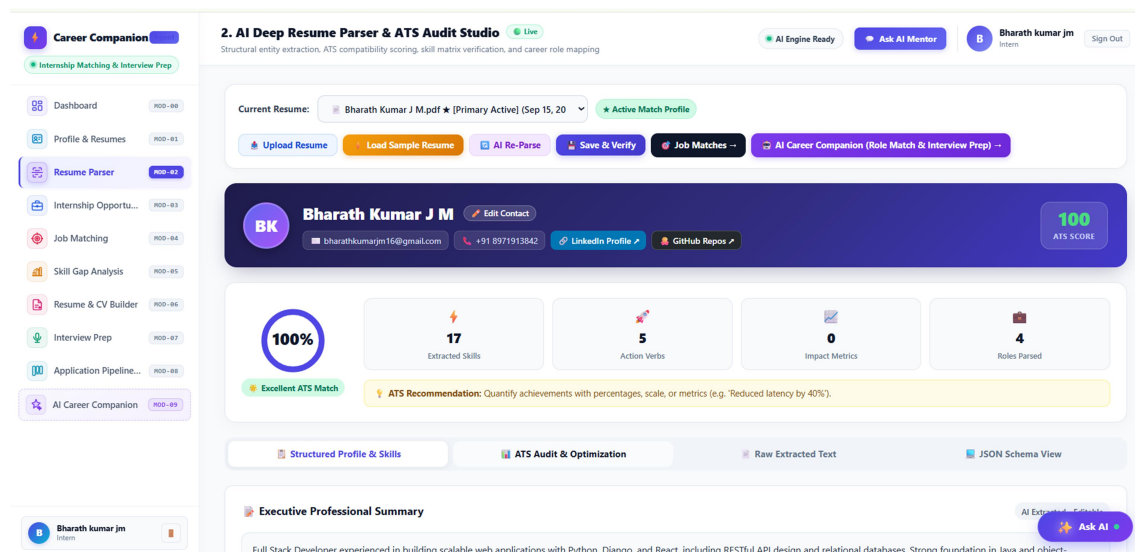
ATS Score & Recommendations

4. Output

Example Result:

- **Overall ATS Score:** 88 / 100
- **Strengths:** Strong technical skills, well-structured sections, and relevant project information.
- **Improvement Recommendation:** Add relevant certifications or strengthen measurable achievements where appropriate.

Screenshot of Resume Parser:



Feature 5: Executive CV Builder & PDF Generator

1. Description

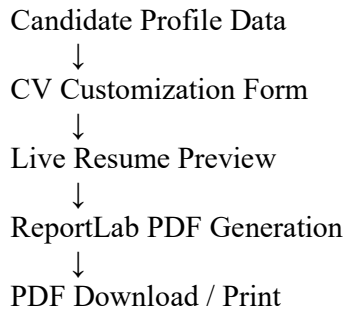
The CV Builder allows candidates to customize their professional resume by editing personal information, summary, skills, education, projects, and experience. The customized CV can be generated and exported as a professional PDF.

2. Implementation & Workflow

1. **Interactive Customization:** React components allow candidates to edit resume information through an interactive form.

2. **Live Preview:** Changes are reflected in the CV preview interface for easier customization.
3. **PDF Generation:** The backend uses **ReportLab** to generate a structured PDF document.
4. **Download Processing:** The generated PDF is returned to the frontend as a downloadable file.
5. **Print Optimization:** Print-specific CSS rules support clean A4-format browser printing.

3. Workflow



4. Output

Generated File:

<Candidate_Name>_Resume_CV.pdf

The generated document contains the candidate's customized professional information in a structured, printable resume format.

Screenshot of Resume & CV Builder:

The screenshot displays the 'Resume & CV Builder' interface. On the left is a sidebar with navigation links: Career Companion, Internship Matching & Interview Prep, Dashboard, Profile & Resumes, Resume Parser, Internship Opportunities, Job Matching, Skill Gap Analysis, Resume & CV Builder (active), Interview Prep, Application Pipeline, and AI Career Companion. The main area is titled '6. Role-Specific Resume & Cover Letter Customization' and includes tabs for 'Tailor Resume for Role', 'AI Cover Letter Generator', and 'Full CV Generator & PDF Exporter'. Below these are buttons for 'Select CV Design Template' (Modern Tech, Harvard ATS, Minimalist), 'AI Auto-Enhance', 'Copy Text', 'Download .TXT', 'Download PDF CV', and 'Print CV'. The 'Customize Resume Content' section has fields for Full Name, Professional Title / Headline, Email Address, Phone Number, Location, and LinkedIn / GitHub. A 'Professional Summary' text area is also present. The 'Programming Languages' section lists Python, Django, React, Java, SQL, JavaScript, HTML5, CSS3, and Ag. On the right is a live preview of the generated resume for 'BHARATH KUMAR JM', Software Engineer Intern, with contact information and sections for Professional Summary, Technical Competencies, and Professional Experience.

10. Testing

Testing was performed throughout the development of the **TalentSprint AI** platform to verify that the implemented features function correctly, securely, and reliably. Testing covered the frontend, backend APIs, database operations, document processing, AI/RAG workflows, authentication, and export functionality.

10.1 Testing Types Performed

1. Functional Testing

Core application functions such as user registration, login, document upload, resume analysis, ATS evaluation, AI chat, application tracking, and PDF generation were tested against their expected behavior.

2. API Testing

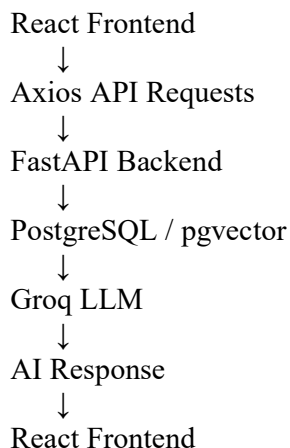
FastAPI REST endpoints were tested using **Swagger UI (/docs)** and automated testing tools. Request validation, response payloads, HTTP status codes, authentication, and protected routes were verified.

3. UI / Usability Testing

The React frontend was tested for correct component rendering, form validation, navigation, notifications, button interactions, document upload behavior, and responsive layout.

4. Integration Testing

End-to-end communication between the major system components was verified:



5. AI & RAG Testing

The RAG pipeline was tested to verify semantic retrieval, context usage, resume-based question answering, and appropriate handling of questions for which relevant candidate information was unavailable.

6. Security Testing

Authentication and authorization mechanisms were tested to ensure protected endpoints require valid JWT credentials and that invalid or missing tokens are rejected.

10.2 Test Cases & Results

Test ID	Test Case	Input / Description	Expected Result	Actual Result	Status
T001	User Registration	Register using valid name, email, and password.	User account should be created successfully.	Account created successfully and registration confirmation displayed.	Pass
T002	Valid Login	Enter registered email and correct password.	JWT token should be generated and user should access the dashboard.	Authentication successful and dashboard accessed.	Pass
T003	Invalid Login	Enter incorrect email or password.	Access should be denied with an authentication error.	Invalid credentials were rejected.	Pass
T004	Valid PDF Upload	Upload a valid PDF resume within the supported size limit.	File should be accepted, processed, and stored.	PDF uploaded and processed successfully.	Pass
T005	Valid DOCX Upload	Upload a valid Word document.	DOCX should be accepted and text extracted successfully.	Document processed and text extracted successfully.	Pass
T006	Unsupported File Type	Upload an unsupported file format.	System should reject the file and display a validation message.	Unsupported file was rejected successfully.	Pass

Test ID	Test Case	Input / Description	Expected Result	Actual Result	Status
T007	File Size Validation	Upload a document exceeding the configured size limit.	Upload should be rejected with a size-limit message.	Oversized document was rejected.	Pass
T008	Resume-Based AI Query	Ask a question related to information contained in the uploaded resume.	Relevant context should be retrieved and a context-aware answer generated.	Relevant candidate information was retrieved and returned.	Pass
T009	Out-of-Context Query	Ask a question unrelated to the uploaded candidate information.	System should avoid presenting unsupported candidate-specific information.	System appropriately indicated that relevant information was unavailable.	Pass
T010	Voice Input	Use the microphone to provide an AI query.	Spoken input should be converted into text in the chat interface.	Voice input was converted into chat text successfully.	Pass
T011	ATS Resume Analysis	Run ATS analysis on an uploaded resume.	System should calculate an ATS score and provide analysis feedback.	ATS score and improvement feedback were generated successfully.	Pass
T012	CV PDF Generation	Customize resume information and generate a PDF.	System should generate and download a valid PDF document.	PDF generated and downloaded successfully.	Pass
T013	Chat Export	Export conversation in supported formats.	Selected export file should contain the conversation content.	Chat content exported successfully.	Pass

Key Observations

1. **Authentication & Security:** Protected API endpoints correctly required valid authentication credentials, while invalid or missing credentials were rejected.

2. **Document Processing:** Supported PDF and DOCX files were successfully uploaded, processed, and prepared for resume analysis and AI-based retrieval.
3. **RAG Functionality:** Relevant candidate context was successfully retrieved through semantic search and supplied to the AI response-generation process.
4. **AI Responses:** Resume-related questions produced context-aware responses, while unsupported candidate-specific information was not presented as known resume data.
5. **Export & PDF Generation:** Generated documents and exported conversations were successfully produced in the supported formats.

Overall Testing Result

The testing phase verified the major functional and integration workflows of the **TalentSprint AI** platform. All documented test cases passed during the project validation phase, resulting in an overall recorded **test pass rate of 100%**.

11. Challenges & Solutions

Developing the **TalentSprint AI** platform as an end-to-end full-stack AI application involved several practical engineering challenges across authentication, document processing, RAG, frontend integration, PDF generation, and voice interaction. The following challenges were identified during development, along with the technical solutions implemented to address them.

Challenge 1: Multi-Column Resume Layouts & Text Extraction

Problem

Modern resumes often contain two-column layouts, sidebars, tables, and skill sections. Basic PDF extraction can merge text from different columns, resulting in incorrectly ordered content and poor-quality text chunks for further processing.

Root Cause

Stream-based PDF extraction may process text according to the underlying document stream rather than the visual arrangement of elements on the page.

Solution

- Integrated **pdfplumber** for improved text extraction from complex PDF layouts.
- Applied text normalization to remove excessive whitespace, unnecessary line breaks, and non-printable characters.
- Used **python-docx** for structured extraction from DOCX documents.
- Prepared the extracted content in a consistent structure before chunking and analysis.

Outcome

The system produced cleaner and more logically structured resume text, improving the quality of subsequent analysis and semantic retrieval.

Challenge 2: LLM Hallucinations & Overly Verbose Responses

Problem

The AI model could sometimes provide generic career advice or infer skills and experience that were not explicitly available in the candidate's uploaded documents.

Root Cause

Without sufficient grounding, an LLM can rely on its general pretrained knowledge rather than limiting its response to the retrieved candidate context.

Solution

- Implemented **strict context-grounding prompts** instructing the model to rely on retrieved candidate information.
- Configured relevant-context retrieval using semantic search and **top-K** results.
- Applied similarity-based filtering to improve the relevance of retrieved content.
- Maintained recent conversational context to support follow-up questions.

Outcome

AI responses became more closely aligned with the candidate's available information and were designed to reduce unsupported assumptions.

Challenge 3: CV Builder PDF Generation & Export Issues

Problem

The CV Builder initially experienced blank PDF output and server-side errors when processing candidate information.

Root Cause

Two main issues were identified:

1. Browser print styles could hide CV elements during PDF or print generation.
2. Candidate data could contain nested lists, dictionaries, project details, and skill categories that were not compatible with a simple flat-text PDF generator.

Solution

- Implemented a server-side PDF generation module using **ReportLab Platypus**.
- Added support for structured candidate data such as skills, experience, education, and project information.
- Developed a dedicated PDF download endpoint for generating and returning the document.
- Updated print-specific CSS rules to keep the CV content visible in A4 format during browser printing.

Outcome

The CV Builder generated complete and properly formatted PDF resumes containing the candidate's available information.

Challenge 4: CORS & JWT Token Persistence

Problem

During frontend-backend integration, some API requests were blocked by browser CORS restrictions, while authentication state could be lost after refreshing the application.

Root Cause

The frontend development origin needed to be explicitly permitted by the FastAPI server, and authentication tokens were not consistently attached to every protected request.

Solution

- Configured FastAPI **CORSMiddleware** with the required development origins.
- Added appropriate support for HTTP methods and authorization headers.
- Implemented an **Axios request interceptor** to retrieve the JWT token from localStorage.
- Automatically attached the token as an Authorization: Bearer <token> header to protected API requests.

Outcome

Frontend-backend communication became more reliable, and authenticated sessions could be maintained across page refreshes during the development environment.

Challenge 5: Speech-to-Text Browser Compatibility

Problem

The voice-input feature could stop listening prematurely during longer technical questions and could behave differently across browsers.

Root Cause

The browser-based **Web Speech API** may apply different silence detection and recognition behavior depending on browser implementation.

Solution

- Added continuous speech-recognition event handling.
- Implemented restart logic when recognition stops unexpectedly.
- Added visual indicators to show when voice input is active.
- Provided standard text-input functionality when speech recognition is unavailable or microphone access is denied.

Outcome

The voice-input feature became more usable for longer queries while maintaining text input as a fallback option.

11.1 Summary Matrix: Challenges vs. Solutions

No.	Challenge Encountered	Technical Root Cause	Implemented Solution	Outcome
1	Multi-column resume text extraction	Text order from complex PDF layouts could become inconsistent	Integrated pdfplumber , normalization, and structured DOCX extraction	Improved text quality and retrieval preparation
2	LLM hallucinations and verbose responses	Responses were not sufficiently constrained to retrieved candidate context	Applied grounding prompts, semantic retrieval, similarity filtering, and conversational context	More context-aware and focused responses
3	CV Builder PDF export issues	Print styles and nested candidate data caused rendering problems	Implemented ReportLab PDF generation and improved print handling	Reliable structured PDF generation
4	CORS errors and JWT persistence	Origin restrictions and inconsistent authorization headers	Configured CORS and implemented Axios JWT interceptors	More reliable authenticated frontend-backend communication
5	Speech-to-text interruptions	Browser-specific Web Speech API behavior	Added continuous recognition handling, restart logic, and fallback input	More stable voice interaction

Overall Result

The challenges encountered during development provided opportunities to improve the **reliability, usability, security, and AI response quality** of the TalentSprint AI platform. Each issue was addressed through targeted changes to the application architecture, backend services, frontend integration, document-processing pipeline, and AI workflow.

12. Final Project Status

The **TalentSprint AI** project has successfully completed its core development activities through four Agile development sprints. The final system integrates secure authentication, document processing, AI-powered resume analysis, RAG-based career assistance, voice interaction, ATS evaluation, CV generation, and conversation export functionality.

12.1 Status Breakdown

Completed Features

- **User Authentication & Session Management:** Implemented secure user registration and login, bcrypt password hashing, JWT-based authentication, protected API routes, and authenticated session handling.
- **Multi-Format Document Ingestion:** Implemented document upload for **PDF, DOCX, and TXT** files with file validation, text extraction, document processing, and storage for further analysis.
- **RAG-Powered AI Career Assistant:** Implemented semantic retrieval using **pgvector** and context-aware AI responses through the **Groq LLM API**, enabling personalized resume and career-related conversations.
- **Speech-to-Text Voice Interaction:** Integrated the **Web Speech API** to allow users to provide voice queries directly through the AI chat interface, with standard text input available as an alternative.
- **Automated ATS Resume Analyzer:** Implemented resume analysis covering important sections, technical keywords, action verbs, readability, and ATS readiness, with an overall score and improvement recommendations.
- **CV Builder & PDF Generation:** Implemented an interactive CV customization module with structured candidate information and server-side PDF generation using **ReportLab**.
- **Multi-Format Conversation Export:** Implemented export functionality for AI conversation sessions and generated content in supported formats such as **PDF, DOCX, and Markdown/text-based formats**.
- **Internship & Career Assistance:** Implemented supporting career features such as internship matching, skill-gap analysis, interview preparation, cover-letter generation, and application tracking.

In Progress

- **System Integration, Performance & Security Testing:** Final verification and optimization of integrated workflows, API performance, security validation, and error handling.
- **Expanded Semantic Knowledge Base:** Enhancement of the internship and technical skill knowledge base to improve the relevance of semantic matching and recommendations.

Pending / Future Roadmap

- **Production Deployment & CI/CD:** Containerize the application using Docker and configure a production deployment and CI/CD workflow.

- **External Job Platform Integration:** Integrate external job and internship platforms to retrieve current opportunities and improve automated matching.
- **Advanced AI Interview Features:** Extend the platform with more advanced mock-interview capabilities and additional AI-driven career assessment features.

12.2 Final Outcome

What has been successfully developed?

A functional full-stack **TalentSprint AI** platform has been developed using a **React + Vite frontend, FastAPI + Python backend, PostgreSQL database, pgvector semantic search, and Groq-based AI inference**. The platform brings document processing, resume analysis, RAG-based AI assistance, ATS evaluation, career guidance, voice interaction, and CV generation into a single system.

Does the application work?

Yes. The core application workflows have been implemented and tested, including authentication, document processing, AI interaction, resume analysis, CV generation, and export functionality. The documented test cases achieved a **100% test pass rate** during the testing phase.

What can the user currently do?

1. Register and securely log in to the platform.
2. Create and maintain a candidate profile with relevant career information.
3. Upload resumes and other supported documents.
4. Analyze resume content and receive ATS-related feedback.
5. Ask career and resume-related questions through the AI assistant.
6. Use voice input for AI queries through the browser.
7. Receive career guidance, skill-gap analysis, internship-related recommendations, and interview preparation support.
8. Customize a CV and generate a downloadable PDF.
9. Export AI conversation content in supported formats.
10. Track internship applications through the application management features.

What remains to be completed?

The **core development scope of the TalentSprint AI platform has been completed**. Remaining work mainly concerns production-level deployment, continuous integration and delivery, expansion of external job/internship integrations, and future enhancements to the AI career and interview capabilities.

12.3 Overall Project Status

Area	Status
Project Setup & Architecture	Completed
User Authentication	Completed
Candidate Profile Management	Completed
Document Upload & Processing	Completed
Resume & ATS Analysis	Completed
RAG & Semantic Search	Completed
AI Career Assistant	Completed
Voice Interaction	Completed
CV Builder & PDF Generation	Completed
Application Tracking	Completed
System Testing	Completed
Production Deployment	Planned
External Job Platform Integration	Planned

Final Status

Overall Project Status: Core Development Completed and Functionally Validated.